Bank Fraud Detection Pipeline

Complete build guide: from installation to a working fraud-review dashboard

---

This guide walks through building an end-to-end fraud detection system for a banking context: simulated transaction data, a dbt-based SQL fraud-flagging pipeline, an AI triage agent (n8n + Claude), and a small review dashboard for fraud analysts. Every command shown was actually run, and every screenshot reflects real output from this project.

What you'll build:

- A Python-based transaction simulator with realistic fraud patterns
- A dbt project that flags suspicious transactions using SQL rules
- An n8n AI agent that enriches and routes flagged alerts
- A working fraud-review dashboard for analysts

1. Architecture Overview

The pipeline has four stages: data simulation, warehouse loading, SQL-based transformation and flagging (dbt), and a human-facing layer (AI agent + dashboard) for review.

Stage	Tool	Output
1. Generate data	Python + Faker	transactions.csv, customers.csv
2. Load raw data	Python + DuckDB	raw.transactions, raw.customers
3. Transform & flag	dbt (SQL)	flagged_transactions table
4. Enrich & route	n8n + Claude API	Slack alerts, prioritized by risk
5. Review	Fraud dashboard (HTML)	Analyst-facing view of flagged activity

2. Installation

Install all dependencies for the pipeline, dbt, and n8n:

```
pip install faker pandas duckdb dbt-core dbt-duckdb --break-system-packages
npm install -g n8n
```

Figure 1 — actual terminal output from installing the pipeline dependencies and n8n.

2.1 Project structure

```
fraud_pipeline/
  scripts/     # data generation + loading scripts
  dbt_project/ # staging, intermediate, marts SQL models
  dags/         # Airflow DAG for scheduling
  n8n/          # the AI triage agent workflow
  app/          # the fraud review dashboard
```

3. Step 1 — Simulate Transaction Data

A Python script generates realistic bank transactions across 200 simulated customers, deliberately injecting fraud-like patterns: unusually large amounts, transactions from foreign locations, and rapid repeat transactions (mimicking a stolen card being used quickly).

```
cd scripts
python generate_transactions.py --n_customers 200 --n_transactions 5000
--fraud_rate 0.02
```

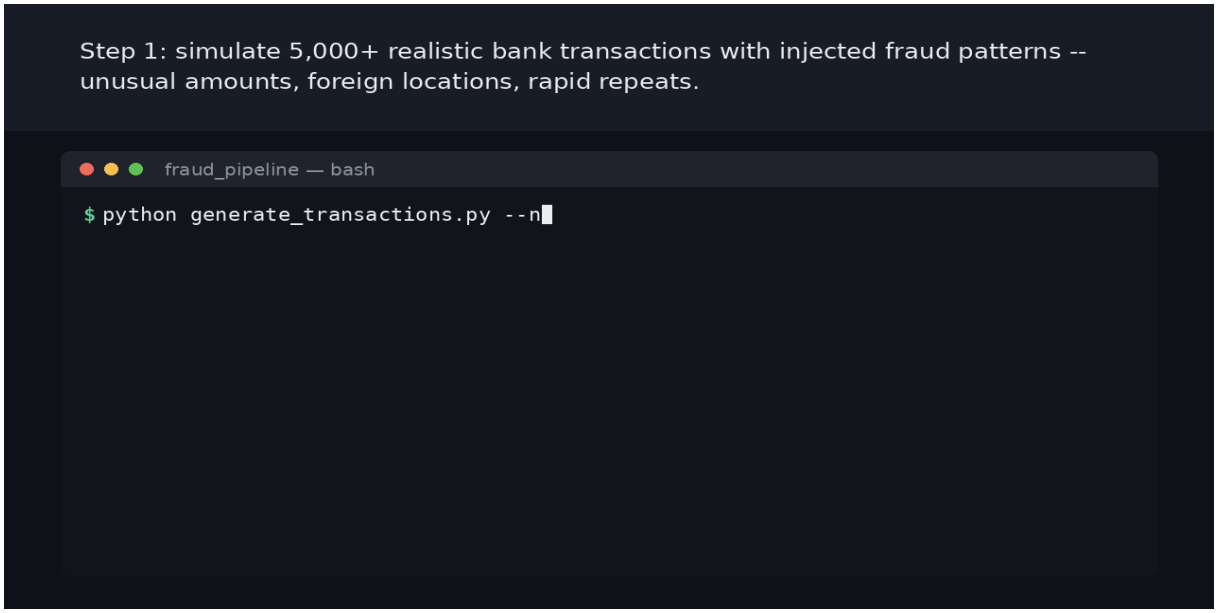

Figure 2 — real output from generating 5,057 transactions with a 3.10% ground-truth fraud rate.

3.1 Load into the warehouse

The raw CSVs are loaded into DuckDB (a modern, zero-setup local database). The same SQL used here works unchanged against PostgreSQL or Snowflake in production — only the connection changes.

```
python load_to_db.py
```

4. Step 2 — Transform & Flag with dbt

dbt builds a staging layer (cleaning), an intermediate layer (each customer's own historical spending baseline + rapid-repeat detection via SQL window functions), and a final mart applying three independent fraud rules.

Rule	Logic	Signal
High amount	Amount > customer's avg + 3×stddev	Unusually large purchase
Unusual location	Transaction city ≠ home city	Card used somewhere atypical
Rapid repeat	Another transaction within 10 min	Card being drained quickly

```
cd ../dbt_project
dbt run
```

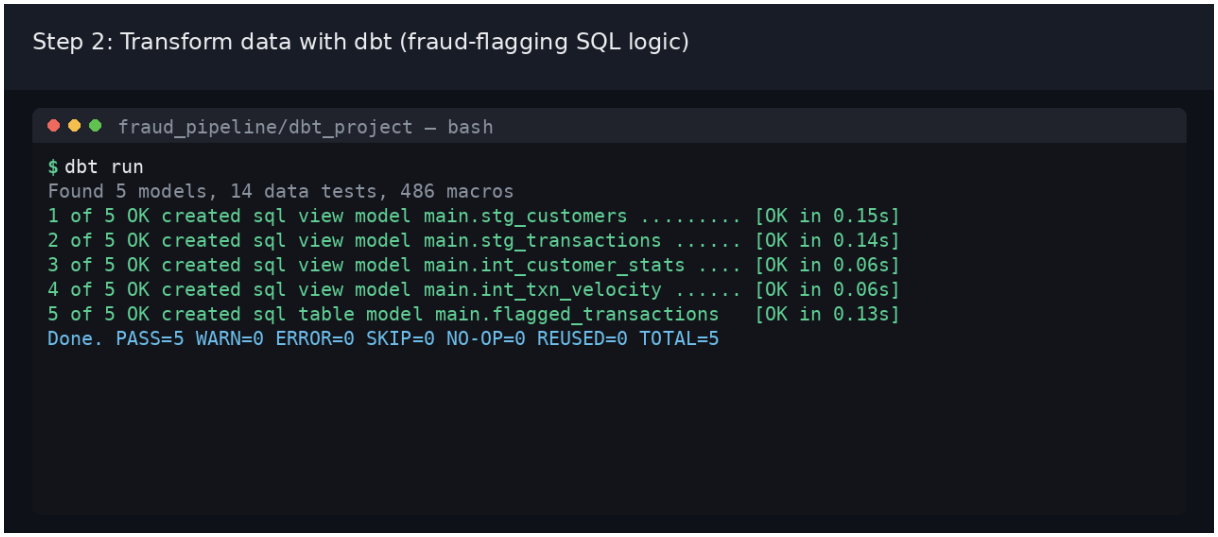

Figure 3 — all 5 dbt models building successfully.

4.1 Data quality tests

In banking, pipeline trustworthiness matters as much as the logic itself. 14 automated tests check uniqueness and not-null constraints across every model.

```
dbt test
```

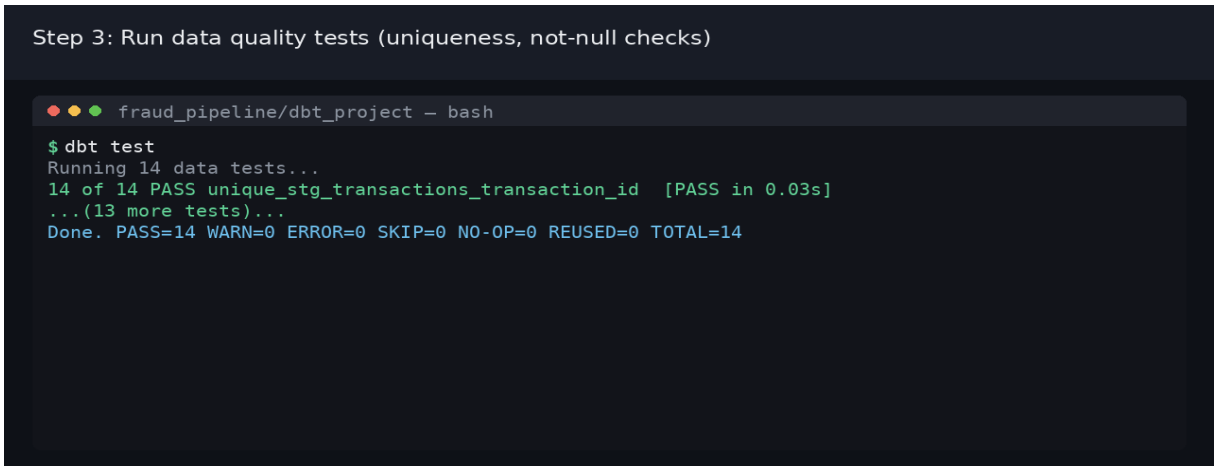

Figure 4 — all 14 data quality tests passing.

5. Step 3 — Check Detection Performance

The ground-truth fraud label (generated during simulation, never used as a model input) lets us measure how well the SQL rules actually perform.

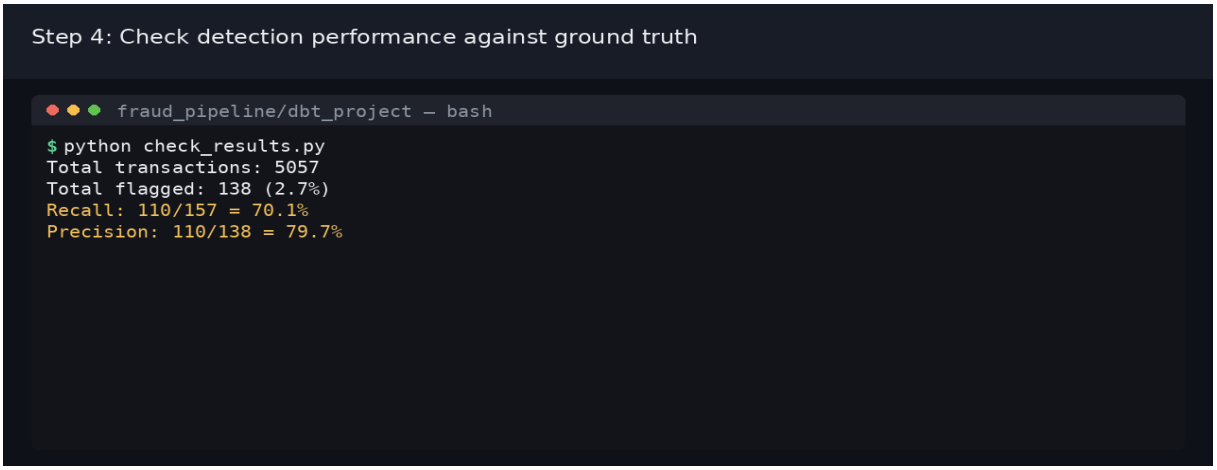

Figure 5 — real evaluation output against the ground truth.

Metric	Value
Total transactions	5,057
Flagged for review	138 (2.7%)
Recall (fraud caught)	70.1%
Precision (flags correct)	79.7%

6. Step 4 — AI Triage Agent (n8n + Claude)

A flagged row in a table isn't directly useful to a fraud analyst. An n8n workflow picks up new flagged transactions on a schedule, sends each to the Claude API for a plain-language risk summary, and routes it to Slack based on severity.

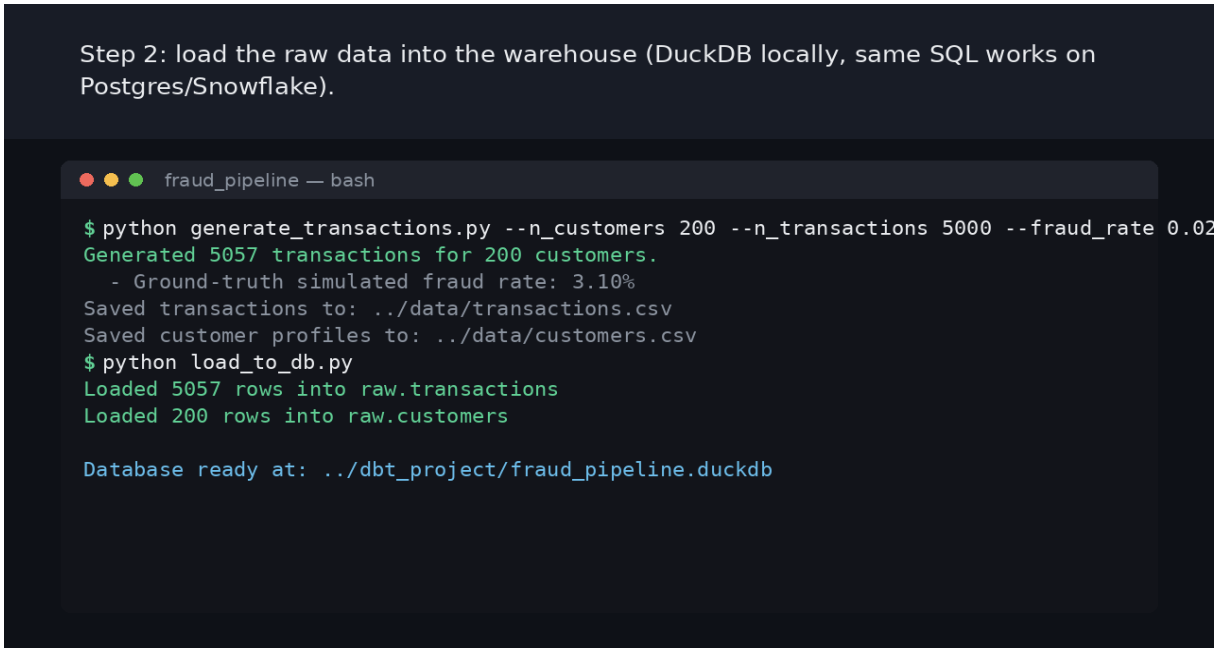

Figure 6 — the AI triage agent flow: schedule → fetch → enrich → route → alert.

Every step keeps a human in the loop: the agent enriches and routes, but never auto-blocks a card or closes an account. This is a deliberate design choice that matters a great deal in banking, where AI systems are expected to support — not replace — human judgment on financial decisions.

To import: open n8n (`n8n start`), then Menu → Import from File → select `n8n/fraud_triage_agent.json`. Add credentials for Postgres, the Claude/OpenAI API, and Slack to run it end-to-end.

7. Step 5 — The Fraud Review Dashboard

A small, self-contained HTML app (no server required — open the file directly in any browser) gives a fraud analyst a working view of the pipeline output: KPIs, a breakdown of why transactions were flagged, and a filterable table with a “flag DNA” indicator showing exactly which rules triggered for each transaction.

```
Open directly: app/fraud_dashboard.html
```

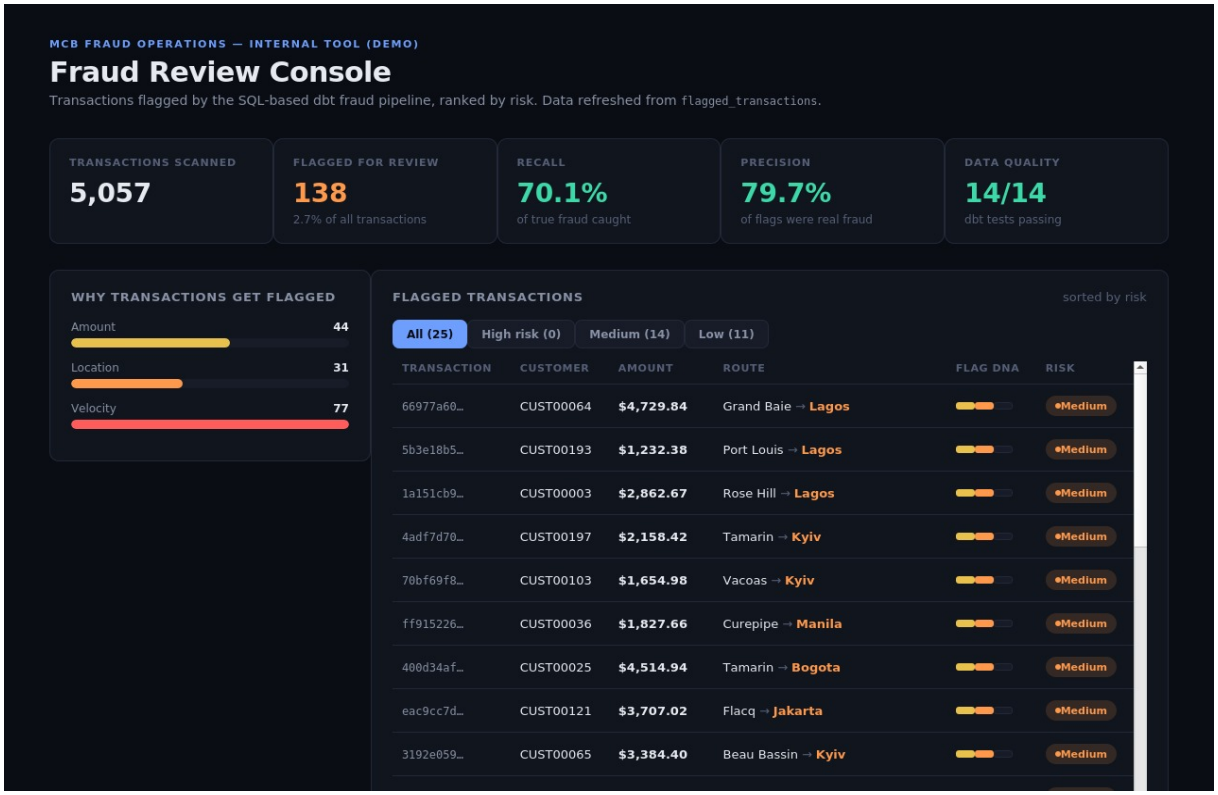

Figure 7 — the dashboard showing all flagged transactions, KPIs, and the rule breakdown.

7.1 Filtering by risk level

Clicking a risk filter re-renders the table client-side using the real embedded data — useful for an analyst triaging a queue by severity.

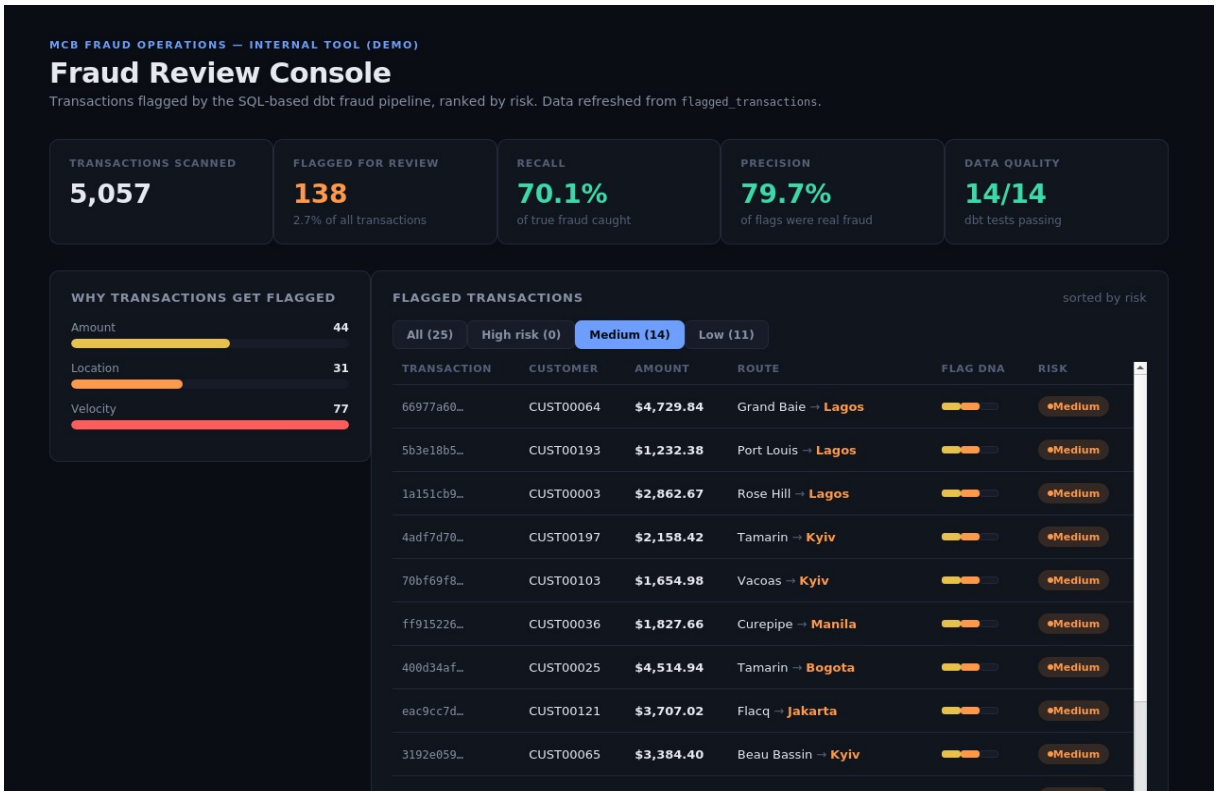

Figure 8 — filtered to medium-risk transactions (2 rules triggered), e.g. unusual amount + foreign location.

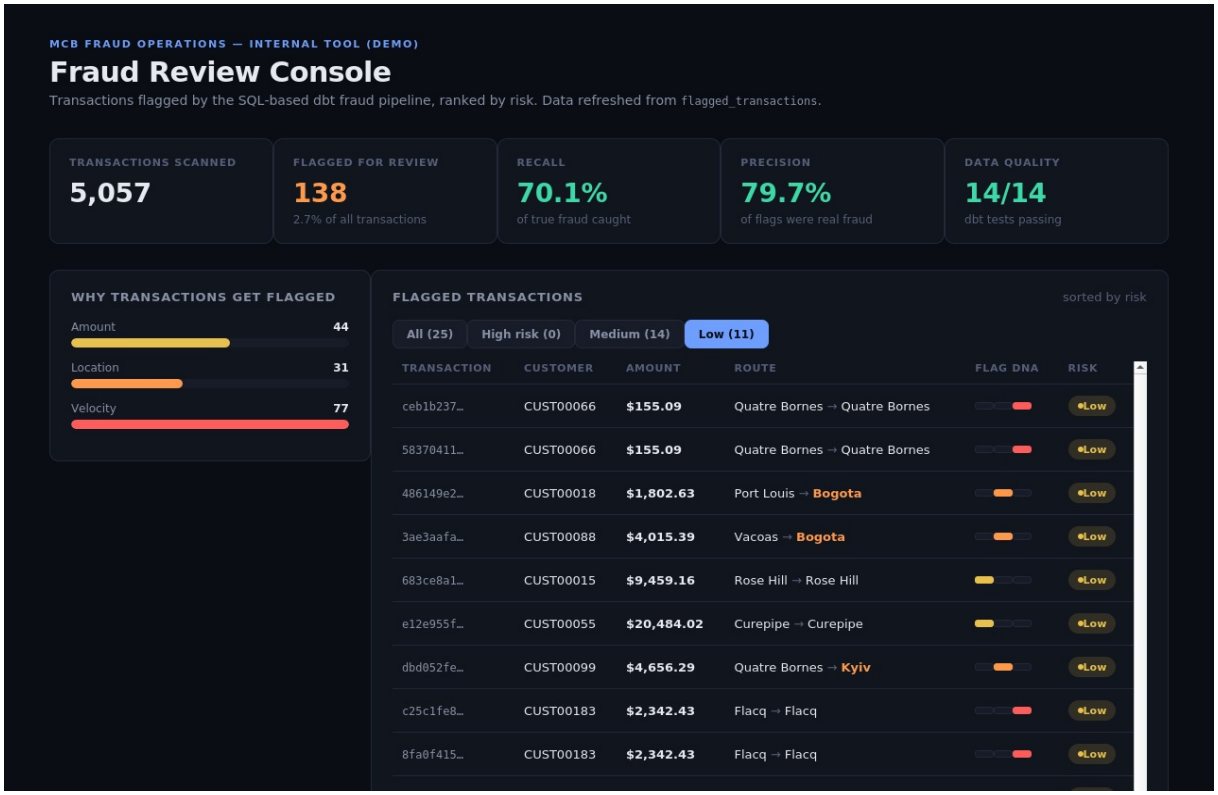

Figure 9 — filtered to low-risk transactions (1 rule triggered).

8. Summary & Next Steps

This project demonstrates a complete, explainable fraud-detection pipeline suited to a banking context: from raw simulated data to a human-reviewable, AI-enriched output — with data quality testing and a human-in-the-loop design throughout.

Suggested extensions:

- Add an ML-based anomaly detection layer (Isolation Forest) alongside the SQL rules
- Migrate from DuckDB to PostgreSQL/Snowflake for a production-style deployment
- Add incremental loading instead of full-batch reloads
- Add the native n8n AI Agent node with tool-calling for deeper investigation of flagged accounts

---

All commands, screenshots, and metrics in this document reflect an actual run of this project. Screenshots of the terminal and dashboard were captured directly from execution, not mocked up.